

Benchmark de performance VASP 6.5.0

Réglage de la parallélisation (NPAR/KPAR) et scaling en nombre de tâches MPI

Pr. John Akapulco
computchem86@gmail.com

22 août 2026

Résumé

Ce rapport présente un benchmark de performance de VASP 6.5.0 sur le cluster de calcul local, réalisé en amont de la campagne de relaxation des polymorphes Al_4C_3 . L'objectif est de déterminer, pour des calculs de type SCF (point unique), la combinaison de paramètres de parallélisation (NPAR, KPAR) et le nombre de tâches MPI qui minimisent le temps CPU, sur trois systèmes de taille et de densité de points $\mathbf{k}$ différentes. Le meilleur compromis obtenu est NPAR=1 avec KPAR égal à 4 ou 8 selon le système, apportant un gain allant de $\times 1.5$ à $\times 7.5$ par rapport à la configuration par défaut (NPAR=KPAR=1), tandis que l'augmentation seule du nombre de tâches MPI sans découpage en points $\mathbf{k}$ sature rapidement.

1 Objectifs

Avant de lancer les relaxations ioniques des polymorphes Al_4C_3 (calculs coûteux, jusqu'à 28 atomes, cellules à 0 et 50 GPa), un benchmark de performance a été mené pour identifier une configuration de parallélisation efficace sur le cluster SLURM local. Deux questions sont posées :

- (a) **Partie A** — à nombre de tâches MPI fixe (`ntasks=40`), quelle combinaison NPAR/KPAR minimise le temps de calcul, et cette combinaison optimale dépend-elle du système traité ?
- (b) **Partie B** — à parallélisation par bandes/points $\mathbf{k}$ désactivée (NPAR=KPAR=1), comment le temps de calcul évolue-t-il avec le nombre de tâches MPI seul (4 à 40) ?

2 Configuration matérielle et logicielle

2.1 Cluster

Les calculs ont été soumis via SLURM (partition `defq`) sur des nœuds mono-nœud (`-nodes=1`), avec deux familles de nœuds au matériel distinct présentes sur le cluster :

Famille	Nœuds	Configuration	CPU logiques
48c_2thread	node01–node08	2 sockets $\times$ 12 cœurs $\times$ 2 threads (SMT)	48
40c_1thread	node09–node16	2 sockets $\times$ 20 cœurs $\times$ 1 thread (SMT off)	40

TABLE 1 – Familles de nœuds de calcul utilisées dans le benchmark.

Le nœud d'exécution n'était pas fixé explicitement dans les scripts de soumission ; l'ordonnanceur SLURM a donc réparti les runs entre les deux familles (colonne `node_family` du fichier de résultats). Ce point est repris comme limite méthodologique en section 7.

2.2 Code et paramètres communs

VASP 6.5.0 (`vasp_std`, mode PAW), MPI Intel (`impi/2021.13`), un thread OpenMP par tâche (`OMP_NUM_THREADS=1`). Tous les calculs sont des points SCF simples (`IBRION=-1`, `NSW=0`, pas de relaxation ionique), avec les paramètres communs suivants :

Paramètre	Valeur
PREC	Accurate
ENCUT	600 eV
EDIFF	10^{-5}
ISMear / SIGMA	0 / 0.05
LWAVE / LCHARG	.FALSE. / .FALSE.

2.3 Systèmes de test

Trois systèmes de taille croissante et de maillages $\mathbf{k}$ très différents (à KSPACING fixé par système) ont été utilisés, afin de vérifier si la configuration optimale dépend du nombre de points $\mathbf{k}$ irréductibles :

Système	Atomes	Formule	KSPACING ($\text{\AA}^{-1}$)	Points $\mathbf{k}$ irréductibles
Ta4C3_7at	7	Ta ₄ C ₃	0.03	189
Ccubic_16at	16	C ₁₆	0.03	105
As4S3_28at	28	As ₁₆ S ₁₂	0.20	27

TABLE 2 – Systèmes de test, densité de maillage $\mathbf{k}$ et nombre de points $\mathbf{k}$ irréductibles résultant (grille de Monkhorst–Pack centrée Γ).

Le KSPACING de **As4S3_28at** est volontairement plus lâche (0.20 contre 0.03) et a été gardé identique sur tous ses runs (Parties A et B), de sorte que les 10 mesures sur ce système restent directement comparables entre elles.

3 Méthodologie

Les runs ont été générés automatiquement (**gen_runs.py**) : POSCAR/POTCAR communs par système, INCAR et script SLURM générés par combinaison, limite de temps de 30 min par job. 22 jobs ont été soumis (IDs SLURM 58341–58362) et suivis jusqu’à completion via une boucle de polling (**wait_loop.sh**, intervalle 60 s).

- **Partie A** (18 runs) : les 3 systèmes $\times$ 6 combinaisons $(\text{NPAR}, \text{KPAR}) \in \{(1, 1), (4, 2), (2, 4), (1, 4), (8, 1), (1, 8)\}$ à **ntasks=40** fixe.
- **Partie B** (4 runs) : le système **As4S3_28at** seul, à **NPAR=KPAR=1** fixe, pour **ntasks** $\in \{4, 8, 16, 24\}$ — complété par le point **ntasks=40**, **NPAR=KPAR=1** déjà mesuré en Partie A.

Le temps CPU (**cpu_time_s**) et le temps écoulé (**elapsed_time_s**) sont extraits de la sortie **time -p** du wrapper **srun**. Chaque configuration n’a été mesurée **qu’une seule fois** (pas de répétition ni d’écart-type ; voir limites, section 7).

4 Résultats

4.1 Partie A — sweep NPAR/KPAR à ntasks=40

4.2 Partie B — scaling en nombre de tâches MPI (**As4S3_28at**, **NPAR=KPAR=1**)

5 Discussion

5.1 Effet de NPAR/KPAR

Sur les trois systèmes, **KPAR=4** ou **KPAR=8** avec **NPAR=1** domine systématiquement les combinaisons à **NPAR>1** (tableau 3, figure 1) :

Système	NPAR	KPAR	Temps CPU (s)	Accélération
Ta4C3_7at	1	1	437.9	1.00
Ta4C3_7at	4	2	268.4	1.63
Ta4C3_7at	2	4	267.6	1.64
Ta4C3_7at	1	4	232.0	1.89
Ta4C3_7at	8	1	369.9	1.18
Ta4C3_7at	1	8	277.5	1.58
Ccubic_16at	1	1	265.7	1.00
Ccubic_16at	4	2	46.1	5.77
Ccubic_16at	2	4	39.2	6.77
Ccubic_16at	1	4	44.2	6.01
Ccubic_16at	8	1	69.7	3.81
Ccubic_16at	1	8	35.5	7.49
As4S3_28at	1	1	372.5	1.00
As4S3_28at	4	2	339.7	1.10
As4S3_28at	2	4	313.2	1.19
As4S3_28at	1	4	240.6	1.55
As4S3_28at	8	1	375.6	0.99
As4S3_28at	1	8	330.8	1.13

TABLE 3 – Temps CPU et accélération par rapport à NPAR=KPAR=1, à `ntasks`=40 fixe. La meilleure combinaison par système est en gras.

ntasks	Temps CPU (s)	Accél. vs 4	Nœud	Famille
4	851.654	1.00	node15	40c_1thread
8	536.230	1.59	node16	40c_1thread
16	539.682	1.58	node04	48c_2thread
24	408.179	2.09	node04	48c_2thread
40	372.519	2.29	node04	48c_2thread

TABLE 4 – Temps CPU en fonction du nombre de tâches MPI, à NPAR=KPAR=1 (pas de parallélisation par points **k** ni par bandes). Le changement de famille de nœud entre `ntasks`=8 et `ntasks`=16 est indiqué (voir section 7).

- **Ccubic_16at** (105 points **k**) : gain maximal $\times 7.49$ avec KPAR=8 — le système a largement assez de points **k** pour remplir 8 groupes.
- **Ta4C3_7at** (189 points **k**) : gain maximal $\times 1.89$ seulement avec KPAR=4, malgré un nombre de points **k** très élevé — ce système, plus petit (7 atomes), est probablement limité par un autre facteur (base d’ondes planes réduite, overhead de communication) plutôt que par le découpage **k**.
- **As4S3_28at** (27 points **k**) : gain maximal $\times 1.55$ avec KPAR=4 ; KPAR=8 est ici *moins* bon ($1.13\times$) que KPAR=4 ($1.55\times$), et NPAR=8/KPAR=1 n’apporte aucun gain ($0.99\times$). Avec seulement 27 points **k**, répartis sur 8 groupes MPI, chaque groupe ne reçoit que 3–4 points **k** et se retrouve avec peu de tâches par groupe (5 rangs/groupe) : l’overhead de communication au sein de chaque groupe l’emporte sur le gain de parallélisation en **k**.

Aucune combinaison à NPAR>1 testée (4/2, 2/4, 8/1) ne bat la meilleure combinaison à NPAR=1 sur un seul des trois systèmes. La parallélisation par bandes (NPAR) semble ici globalement moins efficace que la parallélisation par points **k** (KPAR) pour ces tailles de système et ce nombre

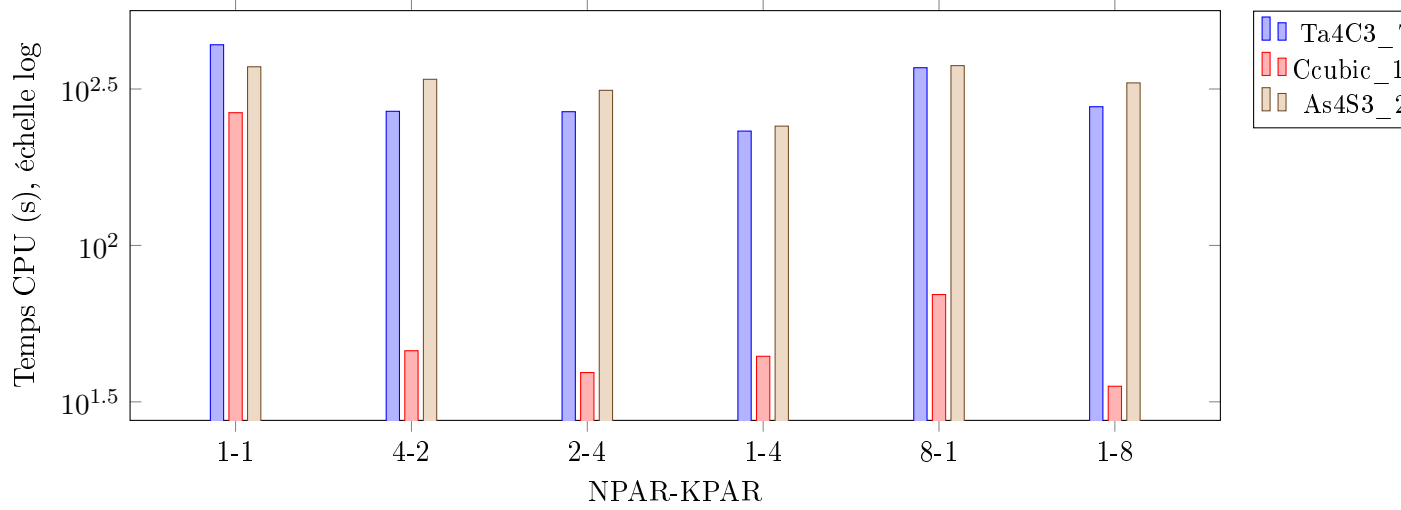


FIGURE 1 – Temps CPU (échelle logarithmique) pour les 6 combinaisons NPAR/KPAR testées, à `ntasks=40`, pour les 3 systèmes. Le gain lié à `KPAR>1` est d'autant plus marqué que le système a beaucoup de points `k` irréductibles.

de tâches.

5.2 Effet du nombre de tâches MPI seul

Sans découpage `k` (`NPAR=KPAR=1`, tableau 4, figure 2), le passage de 4 à 40 tâches ne procure qu'un facteur $\times 2.29$, très loin du $\times 10$ idéal. L'essentiel du décrochage intervient déjà entre `ntasks=4` et `ntasks=8` (efficacité de $\sim 79\%$ en doublant les tâches), puis le gain devient marginal au-delà de 16 tâches. Ceci confirme, en creux, l'intérêt de la Partie A : passer de 40 tâches « brutes » à 40 tâches réparties en `KPAR=4` groupes de points `k` rapporte plus que d'augmenter le nombre de tâches MPI sans structurer le parallélisme.

6 Recommandations pratiques

Pour les calculs de relaxation Al_4C_3 (28 atomes, `ENCUT=600`, cellules à 0 et 50 GPa, `KSPACING=0.03`, donc un nombre de points `k` irréductibles probablement élevé comme `Ta4C3_7at/Ccubic_16at` plutôt que comme `As4S3_28at`) :

- Utiliser `NPAR=1` par défaut, et régler `KPAR` en fonction du nombre de points `k` irréductibles réel du système (`KPAR=4` est un choix sûr et quasi-optimal dans les trois cas testés ; `KPAR=8` n'aide que si le système a suffisamment de points `k` pour remplir 8 groupes sans sous-alimenter chaque groupe).
- Ne pas dépasser `ntasks=24-40` sans `KPAR>1` : au-delà, le gain marginal est faible pour des systèmes de taille comparable (16–28 atomes).
- Vérifier le nombre de points `k` irréductibles (ligne "irreducible k-points" de l'OUTCAR) avant de figer `KPAR` pour un nouveau polymorphe, plutôt que de réutiliser aveuglément la config optimale d'un autre système.

C'est la configuration `NPAR=1 / KPAR=4` qui a été retenue dans les scripts de relaxation Al_4C_3 (`vasp6_relax.sh`, cf. `INCAR.relax`) suite à ce benchmark.

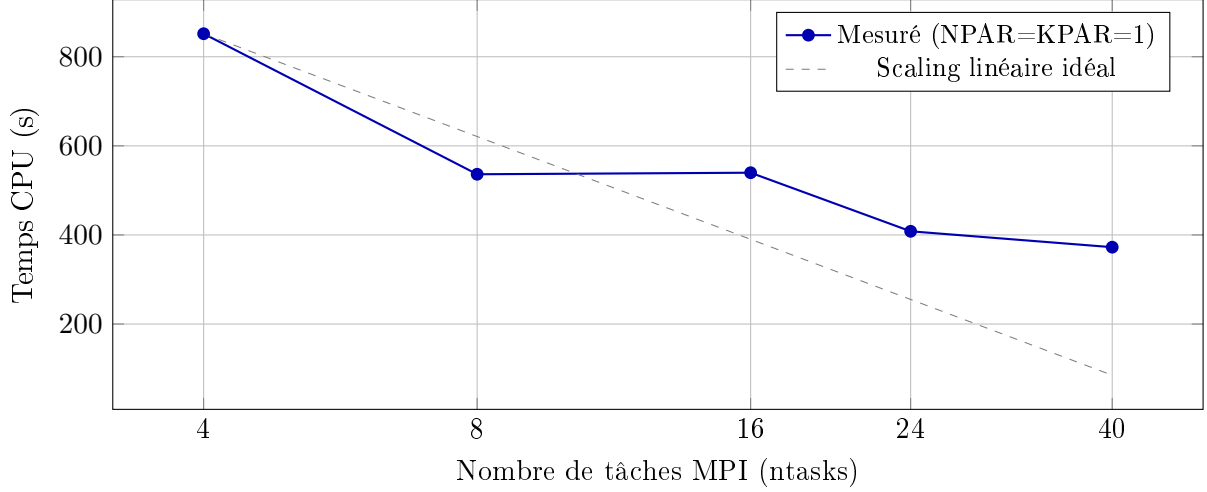


FIGURE 2 – Temps CPU vs nombre de tâches MPI pour `As4S3_28at`, comparé au scaling linéaire idéal extrapolé depuis `ntasks=4`. Le décrochage est net dès `ntasks=8` : sans découpage en points `k` (`KPAR=1`), le système (27 points `k` irréductibles) sature bien avant `ntasks=40`.

7 Limites de l'étude

- **Une seule mesure par configuration** : aucune répétition n'a été effectuée, donc aucune barre d'incertitude n'est disponible. La variabilité liée à la charge du nœud ou au réseau n'est pas quantifiée.
- **Hétérogénéité matérielle non contrôlée en Partie B** : les points `ntasks=4` et `ntasks=8` ont tourné sur des nœuds sans SMT (`40c_1thread`), tandis que `ntasks=16`, `24`, `40` ont tourné sur un nœud avec SMT actif (`48c_2thread`). La comparaison brute entre ces deux groupes de points n'isole donc pas purement l'effet du nombre de tâches.
- **Systèmes non représentatifs à 100%** des polymorphes Al_4C_3 : les trois systèmes de test diffèrent en composition chimique et en `KSPACING`. Les conclusions sur la meilleure combinaison `NPAR/KPAR` sont extrapolées par analogie sur le nombre de points `k`, pas vérifiées directement sur Al_4C_3 .
- Les temps rapportés sont des temps CPU/écoulés pour un unique point SCF (`IBRION=-1`), non pour une relaxation ionique complète ; l'INCAR de production diffère légèrement (`IBRION=2`, `ISIF=8`, `EDIFFG=-0.01`).

8 Conclusion

Le découpage par points `k` (`KPAR`) est, sur les trois systèmes testés, nettement plus efficace que le découpage par bandes (`NPAR`) ou que la simple augmentation du nombre de tâches MPI. La configuration `NPAR=1 / KPAR=4` constitue un choix robuste par défaut (gain $\times 1.5$ à $\times 6$ selon le système), avec un gain supplémentaire possible via `KPAR=8` uniquement pour les systèmes disposant d'un grand nombre de points `k` irréductibles. Ce réglage a été adopté pour la campagne de relaxation des polymorphes Al_4C_3 .

A Détails de soumission

22 jobs soumis sur SLURM (partition `defq`), IDs 58341 à 58362, générés et suivis via `benchmarks/scf_speed_` et `wait_loop.sh`. Données brutes : `benchmarks/scf_speed_test/results_cpu_time.csv`.